KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Öğrenci Ders Notu

HAFTA 03

Nesne Yöneliminin Dört Temel İlkesi

Kapsülleme · Soyutlama · Miras · Çok Biçimlilik — bütünlüklü bir bakış

ÖZET

İlk iki haftada **tek bir sınıfı** doğru yazmayı öğrendik: alanlar, metotlar, yapıcı, `this`, `private` alan ve getter/setter. Bu hafta kamerayı geriye çekiyoruz. Nesne yönelimini bir *dil özelliği* yığını olmaktan çıkarıp **dört ilkeye** dayanan bir tasarım yaklaşımı olarak göreceğiz: **kapsülleme**, **soyutlama**, **miras** ve **çok biçimlilik**. Bu hafta derinlik değil **bütünlük** haftasıdır: dört ilkeyi tek bir çalışan sistem üzerinde bir arada göreceğiz, her birinin ayrıntısını ise ilgili haftasına bırakacağız.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

İçindekiler

1. Bu Haftanın Yol Haritası ... 3

1.1. Ön Koşul Bilgi Kontrolü ... 3
1.2. Haftanın Zaman Planı ... 4
1.3. Bu Haftanın Kavram Haritası ... 4

2. Motivasyon: Dört Şekil, Üç switch, Bir Sessiz Hata ... 6

2.1. Problem: Birbirine Benzeyen Ama Aynı Olmayan Varlıklar ... 6
2.2. Programı Çalıştıralım ... 7
2.3. Bu Tasarımın Üç Kusuru ... 8
2.4. Sorunun Kökeni ... 8

3. Dört İlkenin Tanımı ve Ortak Sahne ... 10

3.1. Tanımlar ... 10
3.2. Ortak Sahne: Sekil Üst Sınıfı ... 10
3.3. Alt Sınıflar ... 12
3.4. Yeni Bir Tür Eklemek ... 13

4. Derinleşme: Dört İlke Tek Tek ... 15

4.1. Kapsülleme: Veriye Giden Tek Kapı ... 15
4.2. Soyutlama: “Ne” ile “Nasıl”ın Ayrılması ... 18
4.3. Miras: Ortak Olanı Bir Kez Yazmak ... 21
4.4. Çok Biçimlilik: Aynı Çağrı, Farklı Gövde ... 22
4.5. Bir Referansın İki Türü ... 24
4.6. Kalıtım mı, Arayüz mü? (IS-A / CAN-DO) ... 25
4.7. Dört İlke Aynı Anda ... 26

5. C'den Java'ya Köprü ... 27

6. Sık Yapılan Hatalar ... 29

6.1. Kapsüllemeyi Yalnızca Alan Gizleme Sanmak ... 29
6.2. @Override İşaretleyicisini Yazmamak ... 29
6.3. Soyut Sınıftan Nesne Üretmeye Çalışmak ... 30
6.4. Kalıtımı Kod Tekrarını Azaltmak İçin Zorlamak ... 30
6.5. Üst Tür Referansı ile Alt Tür Üyesine Erişmeye Çalışmak ... 30
6.6. Alt Sınıfta super(...) Çağrısını Unutmak ... 31

7. İyi Tasarım İlkeleri ... 32

8. Bir Sonraki Haftaya Köprü ... 33

9. Haftanın Özeti ... 34

10. Alıştırmalar ... 35

10.1. Kavram Soruları ... 35
10.2. Kod Okuma ve Hata Bulma ... 35
10.3. Programlama ... 36

11. Öz Değerlendirme ... 37

Öz Değerlendirme Kontrol Listesi 37
12. Kaynaklar 38
12.1. Ders Kitapları 38
12.2. Çevrimiçi Kaynaklar 38
12.3. Bu Haftanın Kod Dosyaları 38
12.4. İletişim 39

1. Bu Haftanın Yol Haritası

Hafta 1'de nesne yönelimli düşünmenin temel sorusunu sorduk: *"Bu problemde hangi varlıklar var, her biri neyi bilir ve ne yapabilir?"* Hafta 2'de bu varlıkların **doğru doğmasını** sağladık: yapıcı metot, `this`, aşırı yükleme ve `private` alan + getter/setter düzeni. İki hafta sonunda elimizde **tek başına düzgün çalışan bir sınıf** var.

Ancak gerçek yazılımlar tek sınıftan oluşmaz. Bir okul yönetim sisteminde öğrenci de vardır personel de; bir tahsilat sisteminde kredi kartı da vardır havale de; bir çizim programında daire de vardır kare de. **Birbirine benzeyen ama aynı olmayan** varlıklar aynı programda buluştuğunda, Hafta 2'de öğrendiğimiz araçlar tek başına yetmez. Bu hafta yetmediği noktayı somut olarak göreceğiz ve dört ilkeyle nasıl aşıldığını inceleyeceğiz.

Bir uyarıyı baştan yapalım: bu hafta **hiçbir ilke sonuna kadar işlenmeyecektir**. Amacımız dördünü birbirine bağlayan haritayı çıkarmak ve her birinin hangi problemi çözdüğünü kavratmaktır. Kapsüllemenin ayrıntısı Hafta 4'te, mirasınki Hafta 7'de, çok biçimliliğinki Hafta 9'da, soyut sınıf ve arayüzlerinki Hafta 10'da açılacaktır.

> **◎ Bu Bölümün Sonunda Öğrenci**
>
> - Nesne yöneliminin dört temel ilkesini adlandırabilir; her birinin hangi somut problemi çözdüğünü bir cümleyle açıklayabilir.
> - **Kapsüllemenin** yalnızca alanları `private` yapmak olmadığını, veriyi değiştiren **iş kuralını** da sınıfın içine taşımak olduğunu örnekle gösterebilir.
> - **Soyutlamanın** "ne" ile "nasıl"ı ayırdığını; soyut sınıf ve arayüzün bu ayrımı nasıl dile döktüğünü açıklayabilir.
> - **Miras** ilişkisini `extends` ile kurabilir; hangi üyelerin devralındığını ve `IS-A` ilişkisinin ne anlama geldiğini söyleyebilir.
> - **Çok biçimliliği** üst tür referansı + alt tür nesnesi düzeniyle uygulayabilir; aynı çağrının neden farklı gövdeler çalıştırdığını açıklayabilir.
> - Bir referansın **bildirim türü** (statik) ile **gerçek türü** (dinamik) arasındaki farkı ayırt edebilir ve hangisinin neyi belirlediğini söyleyebilir.
> - Verilen bir kod parçasında hangi ilkenin kullanıldığını **tespit edebilir**.
> - Tip kodu + `switch` yaklaşımının neden kırılgan olduğunu ve tür hiyerarşisiyle nasıl değiştirileceğini gerekçelendirebilir.

1.1. Ön Koşul Bilgi Kontrolü

Aşağıdaki soruların tamamını Hafta 1 ve Hafta 2 bilgisiyle yanıtlayabiliyor olmalısınız. Yanıtlayamadığınız madde varsa laboratuvar öncesinde ilgili bölümü tekrar etmeniz önerilir.

1. Bir sınıf ile o sınıftan üretilen nesne arasındaki fark nedir? `new` tam olarak neyi yapar?
2. Yapıcı metot sıradan bir metottan hangi üç noktada ayrılır?
3. `this.ad = ad;` satırındaki iki `ad` hangi bellek bölgelerinde durur?

4. Bir alanı `private` yapmak neyi engeller? `getter`/`setter` neden yazılır?
5. `toString()` metodunu ezdiğinizde `System.out.println(nesne)` çağrısı ne yapar?

1.2. Haftanın Zaman Planı

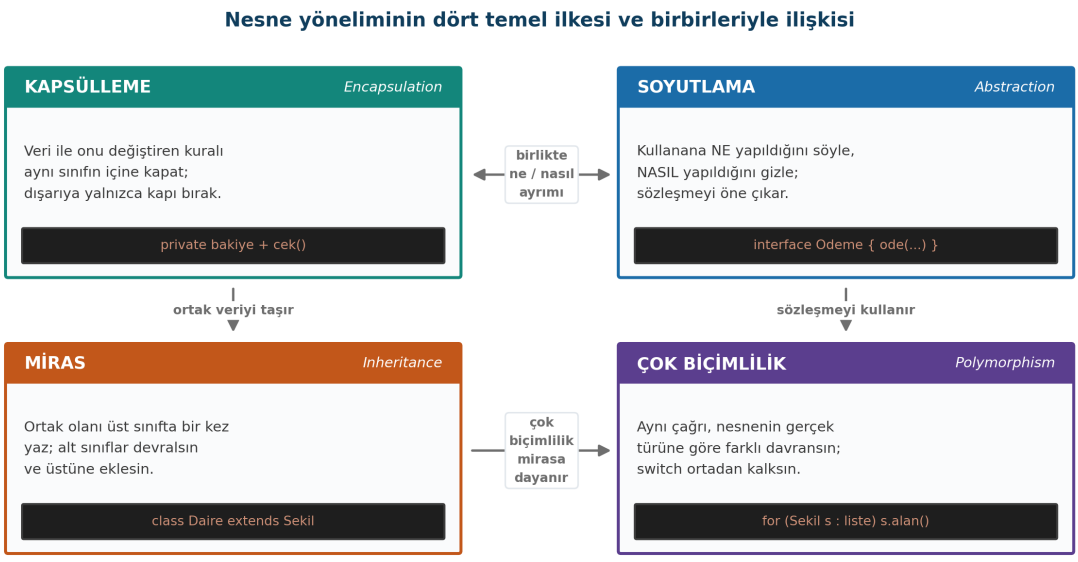
Süre	Etkinlik	İçerik
Teori 1. saat	Ders anlatımı	Tip kodu sorunu, dört ilkenin tanımı, kapsülleme ve soyutlama (Bölüm 2, 3, 4.1 ve 4.2)
Teori 2. saat	Ders anlatımı + tartışma	Miras, çok biçimlilik, statik/dinamik tür, arayüz–kalıtım ayrımı (Bölüm 4.3–4.6, 5 ve 6)
Lab 1. saat	Rehberli uygulama	İlke tespiti ve kapsülleme onarımı (Görev 0, 1 ve 2)
Lab 2. saat	Bireysel uygulama	Miras, çok biçimlilik ve bütünleştirici mini tasarım (Görev 3, 4 ve 5)

Tablo 3.1 — Haftalık 4 saatlik ders yükünün dağılımı

1.3. Bu Haftanın Kavram Haritası

İlke	Cevapladığı soru	Java'daki karşılığı	Hangi bölümde?
Kapsülleme	Veriye kim, hangi kuralla dokunabilir?	`private` alan + `public` metot	Bölüm 4.1
Soyutlama	Kullanana ne söylenir, ne gizlenir?	`abstract` sınıf, `interface`	Bölüm 4.2
Miras	Ortak olan nerede bir kez yazılır?	`extends`, `super`	Bölüm 4.3
Çok biçimlilik	Aynı çağrı nasıl farklı davranır?	Metot ezme + dinamik bağlama	Bölüm 4.4
Statik / dinamik tür	Derleyici ne görür, JVM ne çalıştırır?	Bildirim türü ↔ gerçek tür	Bölüm 4.5
IS-A / CAN-DO	Kalıtım mı, arayüz mü?	`extends` ↔ `implements`	Bölüm 4.6

Tablo 3.2 — Haftanın kavramları ve ele alındıkları bölümler



Şekil 3.1 — Dört temel ilke ve birbirleriyle ilişkisi

> **ⓘ BİLGİ**
>
> Bu dört ilke Java'ya özgü değildir. C++, C#, Python, Kotlin, Swift ve diğer nesne yönelimli dillerin tamamı aynı dört fikri farklı sözdizimiyle sunar. Bu hafta öğrendikleriniz **dil değiştirdiğinizde de geçerli kalacak** kavramlardır.

2. Motivasyon: Dört Şekil, Üç switch, Bir Sessiz Hata

2.1. Problem: Birbirine Benzeyen Ama Aynı Olmayan Varlıklar

Bir çizim programı yazacağız. Programın daireleri, kareleri, dikdörtgenleri ve ileride başka şekilleri tanıması, her birinin alanını ve çevresini hesaplayabilmesi gerekiyor. Hafta 2'de öğrendiğimiz araçlarla bu problemi çözmeye çalışalım: tek bir sınıf yazalım, alanları `private` yapalım, yapıcı ekleyelim. Nesnenin **hangi tür şekil** olduğunu bir metin alanında tutalım. Bu alana **tip kodu** (type code) denir.

SekilKotu.java

```
1  public class SekilKotu {
2
3      // Tip kodu: nesnenin "ne oldugunu" siradan bir metin alani tutuyor.
4      private String tur;
5      private double a;   // daire: yaricap | kare: kenar | dikdortgen: genislik
6      private double b;   // yalnizca dikdortgende anlamli; digerlerinde olu alan
7
8      public SekilKotu(String tur, double a, double b) {
9          this.tur = tur;
10         this.a = a;
11         this.b = b;
12     }
```

Kod 3.1 — Tip kodu yaklaşımı — alanlar ve yapıcı

Alanlara dikkat edin. `a` ve `b` alanlarının **ne anlama geldiği nesnenin türüne göre değişir**: daire için `a` yarıçaptır, kare için kenardır, dikdörtgen için genişliktir. `b` alanı ise yalnızca dikdörtgende anlamlıdır; diğer türlerde bellekte yer kaplar ama hiçbir işe yaramaz. Böyle alanlara **ölü alan** denir ve tasarımın bozulduğunun ilk işaretidir.

Şimdi hesap metotlarını yazalım. Her metot, nesnenin türüne bakıp doğru formülü seçmek zorundadır:

SekilKotu.java

```
1      public double alan() {
2          switch (tur) {
3              case "daire":      return Math.PI * a * a;
4              case "kare":       return a * a;
5              case "dikdortgen": return a * b;
6              default:           return 0.0;   // sessiz yanlis sonuc
7          }
8      }
9
10     public double cevre() {
11         switch (tur) {
12             case "daire":      return 2 * Math.PI * a;
13             case "kare":       return 4 * a;
14             case "dikdortgen": return 2 * (a + b);
15             default:           return 0.0;
16         }
17     }
18
19     public String tanit() {
```

```
        switch (tur) {
            case "daire":      return "Daire(r=" + a + ")";
            case "kare":       return "Kare(k=" + a + ")";
            case "dikdortgen": return "Dikdortgen(" + a + "x" + b + ")";
            default:           return "Bilinmeyen sekil";
        }
    }
```

Kod 3.2 — Aynı `switch` üç kez tekrarlanıyor

Üç metot, üç ayrı `switch`. Üçü de aynı tür listesini tekrar eder. Bu tekrarın bedelini birazdan göreceğiz.

2.2. Programı Çalıştıralım

SekilKotu.java

```
    public static void main(String[] args) {
        SekilKotu[] liste = {
            new SekilKotu("daire", 3.0, 0.0),
            new SekilKotu("kare", 4.0, 0.0),
            new SekilKotu("dikdortgen", 5.0, 2.0),
            new SekilKotu("ucgen", 6.0, 4.0)      // YENI tur: hicbir switch bilmiyor
        };

        System.out.println("--- Tip kodu + switch yaklasimi ---");
        for (SekilKotu s : liste) {
            System.out.printf("%-22s alan=%8.3f  cevre=%8.3f%n",
                              s.tanit(), s.alan(), s.cevre());
        }

        // Anlamsiz nesne uretmek serbest: kare icin ikinci olcu verilebiliyor.
        SekilKotu tuhaf = new SekilKotu("kare", 4.0, 999.0);
        System.out.println();
        System.out.println("Anlamsiz nesne: " + tuhaf.tanit()
                           + " -> b alani 999.0 ama hic kullanilmiyor");
        System.out.println("Ucgen icin alan 0.000 dondu: HATA YOK, sessizce yanlis.");
        System.out.println("Yeni tur eklemek icin UC ayri switch degistirilmeliydi.");
    }
```

Kod 3.3 — Dört şekil, biri yeni bir tür

```
▸ TERMINAL — SekilKotu çıktısı — hesap tablosu
--- Tip kodu + switch yaklasimi ---
Daire(r=3.0)           alan=  28.274  cevre=  18.850
Kare(k=4.0)            alan=  16.000  cevre=  16.000
Dikdortgen(5.0x2.0)    alan=  10.000  cevre=  14.000
Bilinmeyen sekil       alan=   0.000  cevre=   0.000
```

Dördüncü satıra bakın. Programa `"ucgen"` türünde bir şekil verdik. Bu tür hiçbir `switch` içinde tanımlı değil. Program **çökmedi**, **uyarı vermedi**, **derleme hatası üretmedi**. Yalnızca `alan=0.000` ve `cevre=0.000` yazdı.

```
▸ TERMINAL — SekilKotu çıktısı — sorunların özeti
Anlamsiz nesne: Kare(k=4.0) -> b alani 999.0 ama hic kullanilmiyor
Ucgen icin alan 0.000 dondu: HATA YOK, sessizce yanlis.
```

```
Yeni tur eklemek icin UC ayri switch degistirilmeliydi.
```

2.3. Bu Tasarımın Üç Kusuru

#	Kusur	Çıktıdaki kanıt	Neden tehlikeli?
1	Bilinmeyen tür sessizce yanlış sonuç üretir	`Bilinmeyen sekil alan= 0.000`	Hata program çökerek değil, **yanlış rapor** olarak ortaya çıkar; çok geç fark edilir
2	Nesne anlamsız veri taşıyabilir	`Kare(k=4.0) -> b alani 999.0`	Alanın anlamı türe bağlı olduğu için derleyici hiçbir denetim yapamaz
3	Yeni tür eklemek üç ayrı yerde değişiklik gerektirir	`alan()`, `cevre()`, `tanit()` metotlarının üçü de `switch` içerir	Birini güncellemeyi unutmak kolaydır ve derleyici bunu **hatırlatmaz**

Tablo 3.3 — Tip kodu yaklaşımının üç yapısal kusuru

> **▲ DİKKAT**
>
> Üçüncü kusur, yazılım mühendisliğinde **kırılgan tasarım** (fragile design) olarak adlandırılır. Tek bir kavramsal değişikliğin (yeni bir şekil türü) birbirinden uzak birçok noktada eşzamanlı düzeltme gerektirmesi, hata üretmenin en verimli yoludur.

2.4. Sorunun Kökeni

Kodun kendisi kötü yazılmış değil. Alanlar `private`, yapıcı var, isimler anlamlı. Sorun **yapıdadır**: davranış, ait olduğu yerde değil, ondan uzakta duruyor. Dairenin alan formülü “daire” kavramının içinde değil, `SekilKotu` sınıfının `alan()` metodundaki bir `case` etiketinde saklı. Kare için de aynısı. Bilgi dağıldığı için hiçbir tür kendi sorumluluğunu üstlenemiyor.

Nesne yönelimli tasarımın önerdiği çözüm doğrudan bunu düzeltir: **her tür kendi davranışını kendi taşısın.** Bunu yapabilmek için dört ilkeye birden ihtiyacımız var. Aşağıdaki şekil iki tasarımı yan yana koyuyor.



Şekil 3.2 — Aynı problem, iki tasarım: tip kodu ile tür hiyerarşisi

İPUCU

Kodunuzda aynı `switch` veya `if-else` zincirinin **birden çok metotta** tekrar ettiğini fark ederseniz, orada gizlenmiş bir tür hiyerarşisi var demektir. Bu, kod kokusu (code smell) literatüründe *switch statements* adıyla bilinir ve çözümü hemen her zaman çok biçimliliktir.

3. Dört İlkenin Tanımı ve Ortak Sahne

3.1. Tanımlar

Kapsülleme (encapsulation) Bir nesnenin verisini ve o veriyi değiştiren kuralları aynı sınıfın içinde birleştirip, veriye doğrudan erişimi kapatma ilkesidir. Dışarıya yalnızca denetimli bir arayüz (metotlar) açılır. Amaç, nesnenin **her an geçerli bir durumda** kalmasını garanti etmektir.

Soyutlama (abstraction) Kullanana **ne** yapılabileceğini söyleyip **nasıl** yapıldığını gizleme ilkesidir. Bir sözleşme (metot imzaları) öne çıkarılır, gerçekleme ayrıntısı arkada bırakılır. Java'da soyut sınıflar ve arayüzler bu ilkenin dile dökülmüş hâlidir.

Miras (inheritance) Bir sınıfın, başka bir sınıfın alanlarını ve metotlarını devralarak onu genişletmesi ilişkisidir. Ortak olan üst sınıfta bir kez yazılır; alt sınıflar yalnızca kendilerine özgü olanı ekler. Java'da `extends` anahtar sözcüğüyle kurulur.

Çok biçimlilik (polymorphism) Aynı çağrı ifadesinin, çağrıldığı nesnenin **gerçek türüne** göre farklı davranması özelliğidir. `s.alan()` satırı hiç değişmeden, `s` bir daireyi gösteriyorsa daire formülünü, kareyi gösteriyorsa kare formülünü çalıştırır.

> **ⓘ BİLGİ**
>
> Dört ilke birbirinden bağımsız değildir. Kapsülleme veriyi düzenler; soyutlama o düzenin dışarıya nasıl görüneceğini belirler; miras ortak yapıyı taşır; çok biçimlilik ise mirasın üzerine kurulur ve onu kullanışlı kılar. Bu nedenle dördünü tek bir örnek üzerinde birlikte göstermek, ayrı ayrı anlatmaktan daha öğreticidir.

3.2. Ortak Sahne: Sekil Üst Sınıfı

Bölüm 2'deki problemi yeniden, bu kez dört ilkeyi kullanarak çözeceğiz. İlk adım, tüm şekillerin **ortak çekirdeğini** tanımlayan bir üst sınıf yazmaktır. Bu sınıf tek başına üç ilkeyi birden taşır.

Sekil.java

```
1  public abstract class Sekil {
2
3      private final String ad;    // dogumda belirlenir, bir daha degismez
4      private String renk;
5
6      protected Sekil(String ad, String renk) {
7          this.ad = ad;
8          this.renk = gecerliRenk(renk);
9      }
10
11     private static String gecerliRenk(String r) {
12         return (r == null || r.isBlank()) ? "belirsiz" : r;
13     }
14
15     public String getAd() {
16         return ad;
17     }
```

```
18
19      public String getRenk() {
20          return renk;
21      }
22
23      public void setRenk(String renk) {
24          if (renk == null || renk.isBlank()) {
25              System.out.println("Uyari: renk bos birakilamaz; degisiklik yapilmadi.");
26              return;
27          }
28          this.renk = renk;
29      }
```

Kod 3.4 — Sekil üst sınıfı — kapsülleme kısmı

Satır satır çözümleyelim:

Satır	Kod	Hangi ilke?	Ne yapıyor?
1	`public abstract class Sekil`	Soyutlama	`abstract` sözcüğü: bu sınıftan doğrudan nesne üretilemez. `Sekil` bir kavramdır, çizilebilir bir nesne değildir.
3	`private final String ad;`	Kapsülleme	`private`: dışarıdan görünmez. `final`: yapıcıda bir kez atanır, sonra değişmez.
6	`protected Sekil(...)`	Miras	`protected` yapıcı: yalnızca alt sınıfların `super(...)` çağrısıyla erişebileceği bir yapıcı.
23–29	`public void setRenk(...)`	Kapsülleme	Renk değişikliğinin tek kapısı. Boş değer reddedilir; alan geçersiz bir duruma düşemez.

Tablo 3.4 — `Sekil` sınıfının ilk yarısının çözümlemesi (satır numaraları yukarıdaki kod listesine göredir)

Sınıfın ikinci yarısı soyutlama ve miras ilkelerini taşır:

Sekil.java

```
1       // SOYUTLAMA: sozlesme burada, gerceklemesi alt siniflarda.
2       public abstract double alan();
3
4       public abstract double cevre();
5
6       // MIRAS: asagidaki iki metot hicbir alt sinifta tekrar yazilmaz.
7       public String ozet() {
8           return String.format("%-11s renk=%-9s alan=%9.3f  cevre=%9.3f",
9                                ad, renk, alan(), cevre());
10      }
11
12      @Override
13      public String toString() {
14          return ozet();
15      }
16  }
```

Kod 3.5 — Sekil üst sınıfı — soyutlama ve miras kısmı

Listenin 2. satırındaki `abstract double alan();` ifadesinin **gövdesi yoktur**; noktalı virgülle biter. Bu satır bir sözleşmedir: *“benden türeyen her sınıf `alan()` metodunu yazmak zorundadır”*. Sözleşmeyi

doldurmayan bir alt sınıf **derlenmez**. Bölüm 2'deki `default: return 0.0;` sessizliğinin yerini, derleyicinin zorlaması almıştır.

`ozet()` ve `toString()` metotları ise mirasın kazancıdır: bu iki metot **hiçbir alt sınıfta yeniden yazılmayacak**, üç şekil türü de bunları olduğu gibi kullanacaktır.

> ⓘ **BİLGİ**
>
> `ozet()` metodunun içinde `alan()` çağrısı var — ama `Sekil` sınıfında `alan()` metodunun gövdesi yok. Bu nasıl derlenir? Çünkü derleyici **imzanın var olduğunu** bilir; hangi gövdenin çalışacağına çalışma zamanında karar verilir. Bu, çok biçimliliğin çekirdeğidir ve Bölüm 4.4'te ayrıntılandırılacaktır.

3.3. Alt Sınıflar

Şimdi somut şekilleri yazalım. Her biri yalnızca **kendine özgü** olanı içerir; ortak olan hiçbir şeyi tekrarlamaz.

Daire.java

```java
public class Daire extends Sekil {

    private final double yaricap;

    public Daire(double yaricap, String renk) {
        super("Daire", renk);              // once ust sinifin yapicisi
        if (yaricap <= 0) {
            System.out.println("Uyari: yaricap pozitif olmalidir; 1.0 kullanildi.");
            this.yaricap = 1.0;
        } else {
            this.yaricap = yaricap;
        }
    }

    public double getYaricap() {
        return yaricap;
    }

    @Override
    public double alan() {
        return Math.PI * yaricap * yaricap;
    }

    @Override
    public double cevre() {
        return 2 * Math.PI * yaricap;
    }
}
```

Kod 3.6 — Daire — Sekil'in birinci alt sınıfı

`extends Sekil` ifadesi miras ilişkisini kurar. `super("Daire", renk);` satırı, Hafta 2'de öğrendiğiniz yapıcı zincirinin bir üst sınıfa uzanmış hâlidir: önce üst sınıfın yapıcısı çalışır, sonra alt sınıfın gövdesi.

`@Override` işaretleyicisi ise derleyiciye *"bu metot üst sınıftaki bir metodu eziyor"* der; üst sınıfta böyle bir metot yoksa derleme hatası alırsınız.

Kare.java

```java
public class Kare extends Sekil {

    private final double kenar;

    public Kare(double kenar, String renk) {
        super("Kare", renk);
        if (kenar <= 0) {
            System.out.println("Uyari: kenar pozitif olmalidir; 1.0 kullanildi.");
            this.kenar = 1.0;
        } else {
            this.kenar = kenar;
        }
    }

    public double getKenar() {
        return kenar;
    }

    @Override
    public double alan() {
        return kenar * kenar;
    }

    @Override
    public double cevre() {
        return 4 * kenar;
    }
}
```

Kod 3.7 — Kare — aynı sözleşme, farklı formül

İki sınıfı yan yana koyduğunuzda yapının aynı, formülün farklı olduğunu görürsünüz. Sözleşme aynı, gerçekleme farklı — soyutlamanın tanımı budur.

3.4. Yeni Bir Tür Eklemek

Bölüm 2'deki tasarımda yeni bir şekil eklemek üç `switch` bloğunu değiştirmeyi gerektiriyordu. Yeni tasarımda ne oluyor? Sadece **yeni bir dosya yazıyoruz**; var olan hiçbir dosyaya dokunmuyoruz.

Ucgen.java

```java
public class Ucgen extends Sekil {

    private final double taban;
    private final double yukseklik;

    public Ucgen(double taban, double yukseklik, String renk) {
        super("Dik ucgen", renk);
        this.taban = (taban > 0) ? taban : 1.0;
        this.yukseklik = (yukseklik > 0) ? yukseklik : 1.0;
        if (taban <= 0 || yukseklik <= 0) {
            System.out.println("Uyari: olculer pozitif olmalidir; 1.0 kullanildi.");
```

```
        }
    }

    @Override
    public double alan() {
        return taban * yukseklik / 2.0;
    }

    @Override
    public double cevre() {
        return taban + yukseklik + Math.hypot(taban, yukseklik);
    }
}
```

Kod 3.8 — Ucgen — sonradan eklenen tür

Değişiklik gereken dosya	Tip kodu tasarımı	Tür hiyerarşisi tasarımı
Yeni tür sınıfı	yok (yeni sınıf yazılmaz)	**Ucgen.java** (yeni)
`alan()` hesabı	`SekilKotu.java` değişir	değişmez
`cevre()` hesabı	`SekilKotu.java` değişir	değişmez
Tanıtım metni	`SekilKotu.java` değişir	değişmez
Raporlama döngüsü	değişmez	değişmez
Unutma riski	**yüksek** (derleyici uyarmaz)	**yok** (soyut metot doldurulmazsa derlenmez)

Tablo 3.5 — Yeni bir şekil türü eklemenin iki tasarımdaki maliyeti

4. Derinleşme: Dört İlke Tek Tek

Bu bölümde her ilkeyi kendi başına, çalışan bir örnek üzerinde inceleyeceğiz. Amacımız ilkeyi tüketmek değil, **ne işe yaradığını** kesinleştirmektir.

4.1. Kapsülleme: Veriye Giden Tek Kapı

Hafta 2'nin sonunda `private` alan ve `getter`/`setter` düzenini görmüştük. Kapsülleme bundan biraz daha fazlasıdır. Asıl fikir şudur: **veriyi değiştiren iş kuralı, verinin yanında dursun.** Kural sınıfın dışında, çağıran kodda dururken her çağrı yeri kendi kuralını uydurur; kural sınıfın içinde dururken tek bir doğru vardır.

Bir banka hesabı üzerinde görelim. Hesabın bakiyesi vardır ve bu bakiyeye yalnızca üç kuralı geçerek dokunulabilir: tutar pozitif olmalı, bakiye yetmeli, günlük çekim limiti aşılmamalı.

BankaHesabi.java

```java
public class BankaHesabi {

    public static final double GUNLUK_CEKIM_LIMITI = 2000.0;

    private final String iban;          // dogumda belirlenir, degismez
    private final String sahip;
    private double bakiye;              // DISARIYA HIC ACILMAZ
    private double bugunCekilen;

    public BankaHesabi(String iban, String sahip, double acilisBakiyesi) {
        this.iban = iban;
        this.sahip = sahip;
        this.bakiye = (acilisBakiyesi > 0) ? acilisBakiyesi : 0.0;
        this.bugunCekilen = 0.0;
    }

    public String getIban() {
        return iban;
```

Kod 3.9 — BankaHesabi — alanlar ve yapıcı

`GUNLUK_CEKIM_LIMITI` alanı `public static final` bildirilmiştir: dışarıdan okunabilir ama değiştirilemez. Bu, bir **sabittir**; `static` anahtar sözcüğünün ayrıntısı Hafta 6'da işlenecektir.

BankaHesabi.java

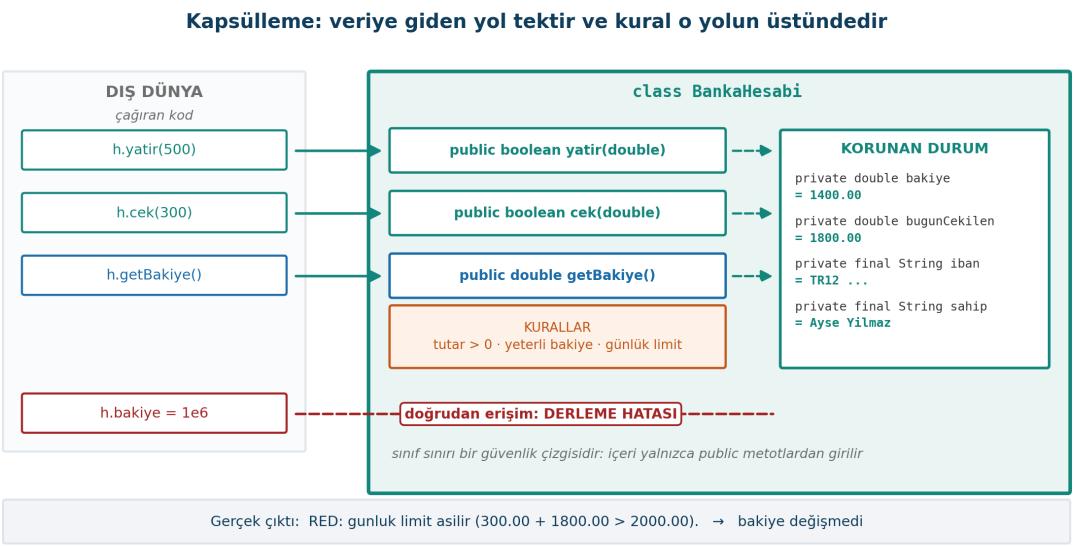
```java
    // Yalnizca OKUMA. setBakiye() BILEREK yazilmamistir.
    public double getBakiye() {
        return bakiye;
    }

    public boolean yatir(double tutar) {
        if (tutar <= 0) {
            System.out.println("  RED: yatirilan tutar pozitif olmalidir.");
            return false;
        }
        bakiye += tutar;
```

```
12          return true;
13      }
14
15      public boolean cek(double tutar) {
16          if (tutar <= 0) {
17              System.out.println("  RED: cekilen tutar pozitif olmalidir.");
18              return false;
19          }
20          if (tutar > bakiye) {
21              System.out.printf("  RED: yetersiz bakiye (istenen %.2f, mevcut %.2f).%n",
22                                tutar, bakiye);
23              return false;
24          }
25          if (bugunCekilen + tutar > GUNLUK_CEKIM_LIMITI) {
26              System.out.printf("  RED: gunluk limit asilir (%.2f + %.2f > %.2f).%n",
27                                bugunCekilen, tutar, GUNLUK_CEKIM_LIMITI);
28              return false;
29          }
30          bakiye -= tutar;
31          bugunCekilen += tutar;
32          return true;
33      }
```

Kod 3.10 — Kural, verinin yanında duruyor

33. satırdaki yorumu özellikle okuyun: **`setBakiye()` metodu bilerek yazılmamıştır.** Bakiyeyi okuyabilirsiniz, ama ona değer atayamazsınız. Bakiye yalnızca `yatir()` ve `cek()` metotlarının işleyişi sonucunda değişir. Bu, kapsüllemenin en güçlü biçimidir.



Şekil 3.3 — Kapsülleme: veriye giden yol tektir ve kural o yolun üstündedir

Programı çalıştıralım:

```
▸ TERMINAL — KapsullemeDemo — geçerli işlemler
Baslangic : Ayse Yilmaz (TR12 0006 1005) bakiye=3000.00 TL

--- Gecerli islemler ---
  yatir(500)  -> true
```

```
  cek(300)    -> true
  Durum: Ayse Yilmaz (TR12 0006 1005) bakiye=3200.00 TL
```

```
▸ TERMINAL — KapsullemeDemo — kuralları zorlayan işlemler
--- Kurallari zorlayan islemler ---
  RED: yatirilan tutar pozitif olmalidir.
  yatir(-100) -> false
  RED: yetersiz bakiye (istenen 99999.00, mevcut 3200.00).
  cek(99999)  -> false
  RED: gunluk limit asilir (300.00 + 1800.00 > 2000.00).
  cek(1800)   -> false
  Durum: Ayse Yilmaz (TR12 0006 1005) bakiye=3200.00 TL

--- Gun degisince ayni istek onaylanir ---
```

Üç ret satırının üçü de **sınıfın içinden** geldi. Çağıran kod hiçbir denetim yapmadı; yapmasına gerek de yok. Kural tek yerdedir, dolayısıyla kuralı değiştirmek gerektiğinde de değiştirilecek tek bir yer vardır.

```
▸ TERMINAL — KapsullemeDemo — gün değişince aynı istek onaylanır
  cek(1800)   -> true
  Durum: Ayse Yilmaz (TR12 0006 1005) bakiye=1400.00 TL

--- Kapsulleme neyi ENGELLEDI? ---
  h.bakiye = 1000000;  satiri DERLENMEZ (private alan).
```

Kapsülleme düzeyi	Ne yapılır?	Ne kazanılır?
Düzey 0 — açık alan	`public double bakiye;`	Hiçbir şey. Her yerden her değer yazılabilir.
Düzey 1 — private + getter/setter	`private` alan, `getBakiye()`, `setBakiye()`	Erişim tek noktadan geçer; ancak kural hâlâ yazılmamıştır.
Düzey 2 — doğrulamalı setter	`setBakiye()` içinde `if (deger < 0) return;`	Geçersiz değer engellenir. Yeterli olduğu durumlar vardır.
Düzey 3 — setter yok, iş metodu var	`yatir()`, `cek()`; `setBakiye()` yazılmaz	Alan yalnızca **anlamlı iş işlemleriyle** değişir. En güçlü biçim.

Tablo 3.6 — Kapsüllemenin dört düzeyi

> ✓ **İPUCU**
>
> Bir alan için `setter` yazmadan önce şu soruyu sorun: *"Bu alanı dışarıdan keyfî bir değere ayarlamak anlamlı bir iş işlemi midir?"* Yanıt hayırsa `setter` yazmayın; onun yerine gerçek iş işlemini adlandıran bir metot yazın. `hesap.setBakiye(1700)` anlamsızdır; `hesap.yatir(500)` anlamlıdır.

4.2. Soyutlama: “Ne” ile “Nasıl”ın Ayrılması

Soyutlama günlük hayatta sürekli kullandığımız bir şeydir. Bir otomobili kullanmak için motorun içinde ne olduğunu bilmeniz gerekmez; direksiyon, pedallar ve vites bir **arayüzdür**. Bu arayüz *ne* yapabileceğinizi söyler, *nasıl* olduğunu gizler. Motor benzinli mi elektrikli mi — kullanım aynıdır.

Yazılımda bu ayrımı ifade etmenin iki yolu vardır: **soyut sınıf** ve **arayüz**. `Sekil` sınıfında soyut sınıfı gördük. Şimdi arayüzü görelim.

Odeme.java

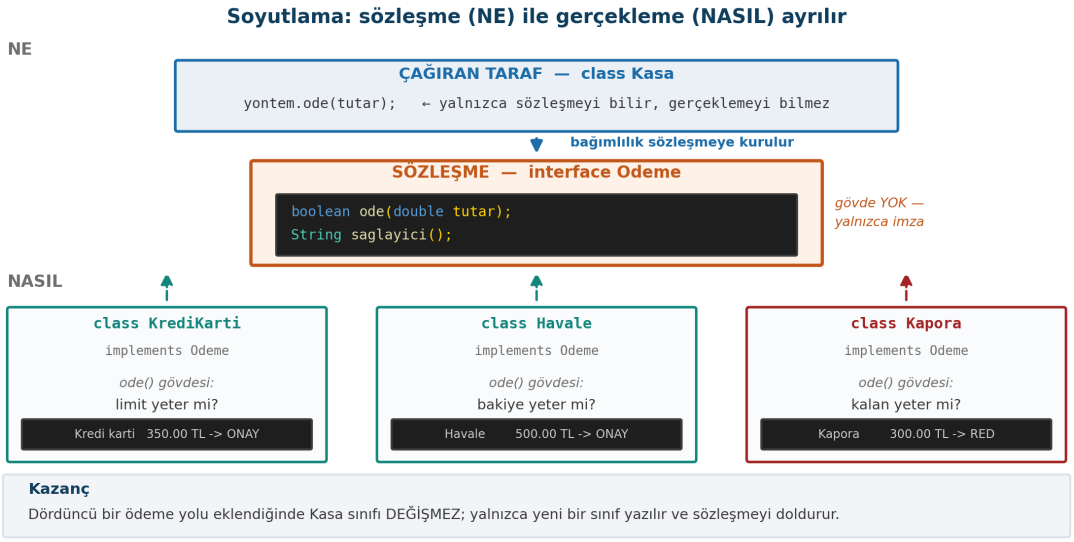
```java
public interface Odeme {

    /** Verilen tutari tahsil etmeye calisir; basarili ise true doner. */
    boolean ode(double tutar);

    /** Odeme saglayicisinin gorunen adi. */
    String saglayici();
}
```

Kod 3.11 — Odeme arayüzü — yalnızca sözleşme

Arayüzde **hiçbir gövde yoktur**, yalnızca imzalar vardır. Bu dosya *“ödeme yapılabilen her şey şu iki metoda sahiptir”* der ve durur. Nasıl yapıldığı ilgilendirmez.



Şekil 3.4 — Soyutlama: sözleşme (NE) ile gerçekleme (NASIL) ayrılır

Sözleşmeyi kullanan taraf, `Kasa` sınıfıdır:

OdemeDemo.java

```java
/** Odeme sozlesmesini bilen; gerceklemeleri bilmeyen cagiran taraf. */
class Kasa {
    static void tahsilEt(Odeme yontem, double tutar) {
        boolean sonuc = yontem.ode(tutar);
        System.out.printf("  %-14s %8.2f TL -> %s%n",
                          yontem.saglayici(), tutar, sonuc ? "ONAY" : "RED");
    }
```

```
8  }
```

Kod 3.12 — Kasa — yalnızca sözleşmeyi bilen çağıran taraf

`tahsilEt` metodunun parametresi `Odeme` türündedir. Metot gövdesinde `KrediKarti`, `Havale` veya `Kapora` sözcükleri **hiç geçmez**. Metot yalnızca sözleşmeye bağımlıdır; gerçeklemelerin varlığından haberi bile yoktur.

Üç gerçekleme, aynı sözleşmeyi üç farklı iç mantıkla doldurur:

OdemeDemo.java

```
1  class KrediKarti implements Odeme {
2      private final String numara;
3      private double limit;
4
5      KrediKarti(String numara, double limit) {
6          this.numara = numara;
7          this.limit = limit;
8      }
9
10     @Override
11     public boolean ode(double tutar) {
12         if (tutar <= 0 || tutar > limit) {
13             return false;
14         }
15         limit -= tutar;
16         return true;
17     }
18
19     @Override
20     public String saglayici() {
21         return "Kredi karti";
22     }
23 }
```

Kod 3.13 — KrediKarti — limit üzerinden gerçekleme

OdemeDemo.java

```
1  class Kapora implements Odeme {
2      private double kalan;
3
4      Kapora(double kalan) {
5          this.kalan = kalan;
6      }
7
8      @Override
9      public boolean ode(double tutar) {
10         if (tutar <= 0 || tutar > kalan) {
11             return false;
12         }
13         kalan -= tutar;
14         return true;
15     }
16
17     @Override
18     public String saglayici() {
```

```
        return "Kapora";
    }
}
```

Kod 3.14 — Kapora — kalan tutar üzerinden gerçekleme

OdemeDemo.java

```
    public static void main(String[] args) {
        Odeme[] yontemler = {
            new KrediKarti("5312 **** **** 4471", 1200.0),
            new Havale("TR33 0001 2222", 600.0),
            new Kapora(250.0)
        };

        double[] sepet = {350.0, 500.0, 300.0};

        System.out.println("--- Kasa: tek dongu, uc farkli odeme yolu ---");
        for (int i = 0; i < yontemler.length; i++) {
            Kasa.tahsilEt(yontemler[i], sepet[i]);
        }

        System.out.println();
        System.out.println("--- Sozlesmeye uymayan tutar ---");
        Kasa.tahsilEt(yontemler[0], -50.0);

        System.out.println();
        System.out.println("Kasa sinifi KrediKarti, Havale, Kapora adlarini hic
gecirmez.");
    }
```

Kod 3.15 — Kasa üç yöntemi tek döngüde işliyor

```
▸ TERMINAL — OdemeDemo çıktısı
--- Kasa: tek dongu, uc farkli odeme yolu ---
  Kredi karti      350.00 TL -> ONAY
  Havale           500.00 TL -> ONAY
  Kapora           300.00 TL -> RED

--- Sozlesmeye uymayan tutar ---
  Kredi karti      -50.00 TL -> RED

Kasa sinifi KrediKarti, Havale, Kapora adlarini hic gecirmez.
```

Kavram	Soyut sınıf (`abstract class`)	Arayüz (`interface`)
Anahtar sözcük	`extends`	`implements`
Kaç tanesi kullanılabilir?	**Bir** üst sınıf	**Birden çok** arayüz
Alan içerebilir mi?	Evet (durum taşıyabilir)	Yalnızca `public static final` sabitler
Gövdeli metot içerebilir mi?	Evet	Evet (`default`/`static`) — Hafta 10
Yapıcısı var mı?	Var (alt sınıf `super()` çağırır)	Yok

Kavram	Soyut sınıf (`abstract class`)	Arayüz (`interface`)
Tipik soru	"Bu **nedir**?" (IS-A)	"Bu **ne yapabilir**?" (CAN-DO)

Tablo 3.7 — Soyut sınıf ile arayüzün karşılaştırması (özet — ayrıntı Hafta 10)

> **ⓘ BİLGİ**
>
> Yazılım tasarımında sık tekrarlanan bir öğüt vardır: **"Gerçeklemeye değil, arayüze programla."** Bu cümlenin somut karşılığı, `Kasa` sınıfının parametre türünü `KrediKarti` değil `Odeme` seçmesidir. Böylece bugün olmayan bir ödeme yöntemi yarın eklendiğinde `Kasa` dosyasına dokunulmaz.

4.3. Miras: Ortak Olanı Bir Kez Yazmak

Miras, bir sınıfın başka bir sınıfın **üyelerini devralması**dır. Devralınan üyeler alt sınıfta yeniden yazılmaz; sanki orada yazılmış gibi kullanılır.



Şekil 3.5 — Miras ağacı: ortak olan yukarıda bir kez yazılır

Üst sınıftaki üye	Alt sınıfta erişilebilir mi?	Neden?
`public` metot	Evet, doğrudan	Herkese açıktır
`protected` üye	Evet, doğrudan	Alt sınıflara ve pakete açıktır
varsayılan (paket) üye	Aynı paketteyse evet	Erişim paket sınırıyla belirlenir
`private` alan	Hayır, doğrudan erişilemez	Ama `public` getter üzerinden okunabilir; alan **devralınır**, yalnızca görünmez
Yapıcı	Devralınmaz	`super(...)` ile **çağrılır**, miras alınmaz
`static` metot	Evet ama ezilemez	Sınıfa aittir, nesneye değil (Hafta 6)

Tablo 3.8 — Miras yoluyla nelerin devralındığı

Somut kanıt: `Ucgen.java` dosyası yalnızca 32 satırdır ve içinde `getAd()`, `getRenk()`, `setRenk()`, `ozet()` veya `toString()` metotlarının hiçbiri yoktur. Yine de aşağıdaki satır sorunsuz çalışır ve `ozet()` metodunu kullanır:

```
▸ TERMINAL — Üç şekil de ozet() metodunu Sekil'den devraldı
--- 1) COK BICIMLILIK: tek dongu, uc farkli davranis ---
  Daire       renk=kirmizi   alan=   28.274  cevre=   18.850
  Kare        renk=mavi      alan=   16.000  cevre=   16.000
  Dik ucgen   renk=yesil     alan=   12.000  cevre=   17.211
  TOPLAM                             56.274
```

> ✖ **SIK YAPILAN HATA**
>
> Miras, kod tekrarını azaltmanın **tek** ve **her zaman doğru** yolu değildir. Yalnızca "alt sınıf gerçekten bir üst sınıf mıdır?" sorusunun yanıtı evet olduğunda kullanılmalıdır. `Araba extends Motor` yanlıştır — araba bir motor değildir, arabanın bir motoru **vardır**. Doğru araç burada mirasa değil **bileşime** (composition) başvurmaktır; ayrıntısı Hafta 7'de.

4.4. Çok Biçimlilik: Aynı Çağrı, Farklı Gövde

Dört ilkenin en çarpıcı sonucu budur. `Sekil` türünden bir dizi kuralım ve içine üç farklı türde nesne koyalım:

SekilDemo.java

```java
    public static void main(String[] args) {
        // Ust tur referansi, alt tur nesnesi: cok bicimliligin can damari.
        Sekil[] sekiller = {
            new Daire(3.0, "kirmizi"),
            new Kare(4.0, "mavi"),
            new Ucgen(6.0, 4.0, "yesil")
        };

        System.out.println("--- 1) COK BICIMLILIK: tek dongu, uc farkli davranis ---");
        rapor(sekiller);
```

Kod 3.16 — Üst tür referansı, alt tür nesnesi

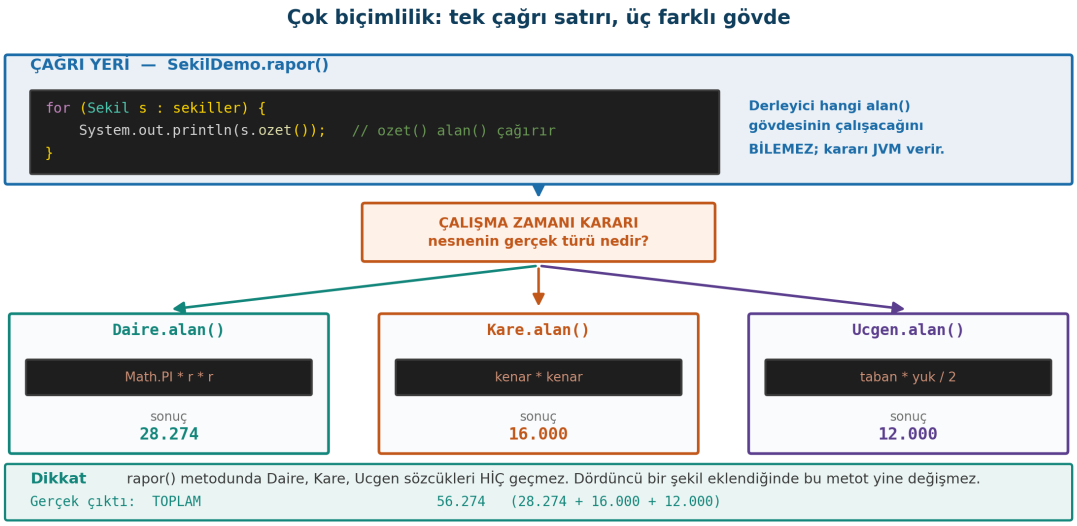
Dizinin türü `Sekil[]` — yani derleyici için dizideki her öğe bir `Sekil`'dir. Ama gerçekte kutuların içinde bir `Daire`, bir `Kare` ve bir `Ucgen` durur. Şimdi raporlama metoduna bakalım:

SekilDemo.java

```java
    // Bu metot Daire, Kare, Ucgen adlarini HIC BILMEZ. Yalnizca Sekil'i bilir.
    private static void rapor(Sekil[] sekiller) {
        double toplamAlan = 0.0;
        for (Sekil s : sekiller) {
            System.out.println("  " + s.ozet());   // dinamik baglama
            toplamAlan += s.alan();
        }
        System.out.printf("  %-11s%21s%9.3f%n", "TOPLAM", "", toplamAlan);
    }
```

Kod 3.17 — rapor() — hiçbir alt sınıf adını geçirmeyen metot

Bu metotta `Daire`, `Kare`, `Ucgen` sözcükleri hiç geçmez; hiçbir `switch` veya `if` yoktur. Buna rağmen her nesne için doğru alan formülü çalışır. Bunu sağlayan mekanizmaya **dinamik bağlama** (dynamic binding) denir.



Şekil 3.6 — Çok biçimlilik: tek çağrı satırı, üç farklı gövde

```
▸ TERMINAL — Dinamik bağlamanın sonucu
--- 1) COK BICIMLILIK: tek dongu, uc farkli davranis ---
  Daire       renk=kirmizi   alan=   28.274  cevre=   18.850
  Kare        renk=mavi      alan=   16.000  cevre=   16.000
  Dik ucgen   renk=yesil     alan=   12.000  cevre=   17.211
  TOPLAM                             56.274
```

Adım	Ne olur?	Ne zaman?
1	Derleyici `s.ozet()` çağrısının **geçerli** olduğunu doğrular (`Sekil` sınıfında `ozet()` bildirilmiş mi?)	Derleme zamanı
2	Derleyici bir "sanal çağrı" komutu üretir; hangi gövdenin çalışacağını **yazmaz**	Derleme zamanı
3	JVM, `s` referansının gösterdiği nesnenin **gerçek sınıfını** okur	Çalışma zamanı
4	O sınıfın metot tablosundan doğru `alan()` gövdesi bulunur ve çalıştırılır	Çalışma zamanı

Tablo 3.9 — Dinamik bağlamanın dört adımı (basitleştirilmiş)

> **ⓘ BİLGİ**
>
> Çok biçimliliğin iki türü vardır. Buraya kadar anlattığımız, **çalışma zamanı çok biçimliliğidir** (metot ezme / overriding). Hafta 2'de gördüğünüz **yapıcı aşırı yükleme** ise derleme zamanı çok biçimliliğidir (overloading): hangi metodun çağrılacağına derleyici, parametre listesine bakarak karar verir. İkisinin ayrımı Hafta 9'da ayrıntılandırılacaktır.

4.5. Bir Referansın İki Türü

Çok biçimliliği doğru kullanabilmek için şu ayrımı netleştirmek gerekir:

```
Sekil s = new Daire(3.0, "kirmizi");
```

Kod 3.18 — Solda bildirim türü, sağda gerçek tür



Şekil 3.7 — Bir referansın iki türü vardır

	Bildirim türü (statik tür)	Gerçek tür (dinamik tür)
Kim belirler?	Programcı, bildirim satırında	`new` ile üretilen nesne
Kim kullanır?	Derleyici	JVM
Neyi belirler?	Hangi üyeler **çağrılabilir**	Hangi gövde **çalışır**
Ne zaman sabitlenir?	Derleme zamanında	Çalışma zamanında
Örnek	`s.alan()` derlenir, `s.getYaricap()` derlenmez	`s.alan()` → `Daire.alan()` gövdesi çalışır

Tablo 3.10 — Statik tür ile dinamik türün karşılaştırması

Bu ayrımın pratik sonucu şudur: üst tür referansıyla, alt sınıfa **özgü** üyelere erişemezsiniz. Erişmek gerekiyorsa önce türü sınamak, sonra dönüşüm yapmak gerekir:

Gorev4_CokBicimlilikDemo.java

```
1          System.out.println("--- 3) instanceof ile guvenli asagi donusum ---");
2          int onur = 0;
3          for (Kisi k : kayitlar) {
4              if (k instanceof Ogrenci) {
5                  Ogrenci o = (Ogrenci) k;          // guvenli, cunku sinandi
6                  if (o.onurListesinde()) {
7                      onur++;
8                  }
9              }
```

```
10          }
11          System.out.println("  Onur listesindeki ogrenci sayisi: " + onur);
```

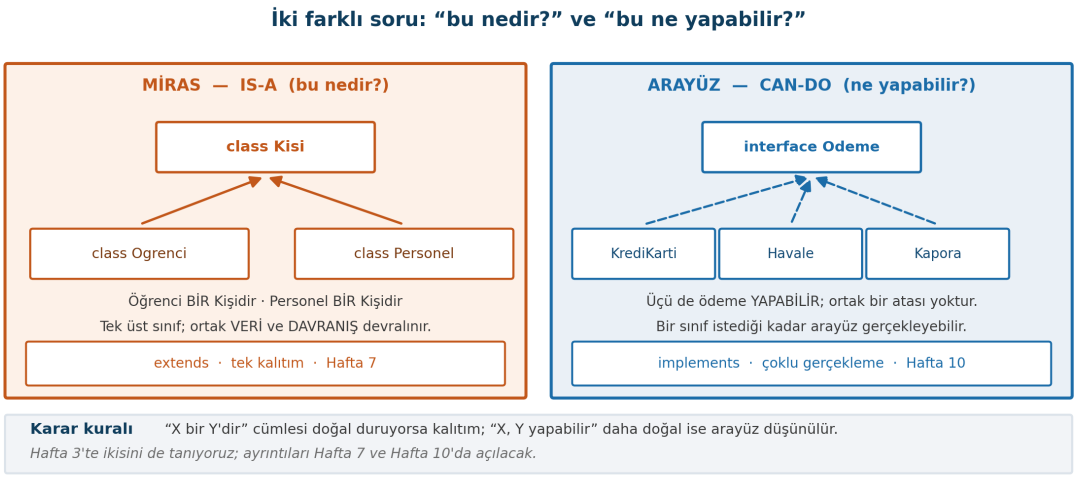
Kod 3.19 — instanceof ile güvenli aşağı dönüşüm

> ✕ **SIK YAPILAN HATA**
>
> `instanceof` sınamasını atlayıp doğrudan `(Ogrenci) k` yazarsanız, `k` gerçekte bir `Personel` ise program `ClassCastException` fırlatarak çöker. Dönüşümden önce **her zaman** sınayın. İstisnaların ayrıntısı Hafta 11'de işlenecektir.

> ✓ **İPUCU**
>
> Kodunuzda çok sayıda `instanceof` sınaması birikiyorsa, bu genellikle çok biçimliliği yeterince kullanmadığınızın işaretidir. `instanceof` zinciri, gizlenmiş bir `switch`tir. Sorulacak soru şudur: *"Bu davranışı sınıfların kendi içine taşıyabilir miyim?"*

4.6. Kalıtım mı, Arayüz mü? (IS-A / CAN-DO)

Miras ve arayüz, ilk bakışta benzer görünür: ikisi de bir tür ilişkisi kurar, ikisi de çok biçimlilik sağlar. Ancak sordukları soru farklıdır.



Şekil 3.8 — İki farklı soru: "bu nedir?" ve "bu ne yapabilir?"

Durum	Doğru araç	Gerekçe
`Ogrenci` ile `Kisi`	Kalıtım (`extends`)	"Öğrenci bir kişidir" doğal bir cümledir; ortak veri (ad, TC) devralınır
`KrediKarti` ile `Havale`	Arayüz (`implements Odeme`)	Ortak bir ata yoktur; yalnızca ortak bir **yetenek** vardır
`Daire` ile `Sekil`	Kalıtım (`extends`)	Ortak veri (ad, renk) ve ortak gövdeli metot (`ozet()`) paylaşılır
Bir sınıf hem karşılaştırılabilir	Çoklu arayüz	Java'da bir sınıf **tek** üst sınıfa, **çok** arayüze sahip olabilir

Durum	Doğru araç	Gerekçe
hem yazdırılabilir olsun		

Tablo 3.11 — Kalıtım ile arayüz arasında seçim yapma

> **▲ DİKKAT**
>
> Java'da bir sınıf **birden çok sınıftan** miras alamaz. Bunun nedeni **elmas problemidir**: iki üst sınıf aynı adlı metodu farklı biçimde yazmışsa, alt sınıf hangisini devralacaktır? Java bu belirsizliği sınıflar için yasaklayarak çözer. Arayüzlerde ise durum karmaşıklaşır ve `default` metotlarla ele alınır — ayrıntısı Hafta 7 ve Hafta 10'da.

4.7. Dört İlke Aynı Anda

`SekilDemo` programı dört ilkeyi birlikte gösterir. Aşağıdaki tablo çıktının hangi satırının hangi ilkeyi kanıtladığını eşleştirir.

```
▸ TERMINAL — SekilDemo — dört ilkenin kanıtı
--- 2) KAPSULLEME: renk yalnizca setter'dan degisir ---
Uyari: renk bos birakilamaz; degisiklik yapilmadi.
  daire.renk = mor
  kare.renk  = mavi

--- 3) SOYUTLAMA: cagiran taraf formulu bilmiyor ---
  En buyuk alanli sekil: Daire

--- 4) MIRAS: ozet() hicbir alt sinifta yazilmadi ---
  Ucgen bir Sekil mi? true
  Sekil bir Ucgen mi? false

--- 5) Gecersiz olcu: nesne yine de gecerli dogar ---
```

Çıktı bölümü	İlke	Ne kanıtlanıyor?
`1) COK BICIMLILIK`	Çok biçimlilik	Tek `for` döngüsü, `switch` yok, üç farklı formül çalıştı
`2) KAPSULLEME`	Kapsülleme	Boş renk reddedildi; `kare.renk` değişmedi (`mavi` kaldı)
`3) SOYUTLAMA`	Soyutlama	`enBuyuk()` metodu hiçbir alan formülünü bilmeden en büyüğü buldu
`4) MIRAS`	Miras	`instanceof Sekil` üç nesne için de `true`; `ozet()` devralındı
`5) Gecersiz olcu`	Kapsülleme	Geçersiz kenar reddedildi, nesne yine de geçerli durumda doğdu

Tablo 3.12 — Çıktı satırları ile ilkelerin eşleştirilmesi

5. C'den Java'ya Köprü

C dilinde nesne yönelimli bir dil desteği yoktur, ancak deneyimli C programcıları dört ilkenin karşılıklarını **elle** kurarlar. Bu bölüm, bildiğiniz C alışkanlıklarının Java'da hangi dil özelliğine dönüştüğünü gösterir.

> **⇄ C DİLİNDEN JAVA'YA KÖPRÜ**
>
> C'de bir modülün iç verisini gizlemek için `.c` dosyasında `static` değişken tanımlar, dışarıya yalnızca başlık dosyasındaki fonksiyonları açarsınız. Bu, elle yapılan **kapsüllemedir**. Java'da aynı işi `private` anahtar sözcüğü dilin kendi kuralı olarak yapar; unutmak veya atlamak mümkün değildir.
>
> - C'de `struct` + fonksiyon işaretçisi tablosu ↔ Java'da sınıf + sanal metot tablosu
> - C'de `void*` ile genel tür ↔ Java'da üst tür referansı (tür güvenli)
> - C'de `switch (tip)` ↔ Java'da çok biçimlilik (derleyici zorlar)

İlke	C dilindeki karşılığı	Java'daki karşılığı	Fark
Kapsülleme	`static` değişken + başlık dosyası disiplini	`private` alan + `public` metot	C'de disiplin **gönüllüdür**; Java'da derleyici **zorlar**
Soyutlama	Fonksiyon işaretçisi alanları içeren `struct`	`interface` / `abstract class`	C'de imza denetimi zayıftır; Java'da sözleşme derleme zamanında denetlenir
Miras	İç içe `struct` (ilk alan olarak taban yapı)	`extends`	C'de işaretçi dönüşümü elle ve tehlikelidir; Java'da tür güvenlidir
Çok biçimlilik	Fonksiyon işaretçisi tablosunu elle kurma	Metot ezme + dinamik bağlama	C'de tablo bakımı programcıya aittir; Java'da JVM yönetir

Tablo 3.13 — Dört ilkenin C ve Java'daki karşılıkları

Aşağıdaki C kodu, Java'daki `Sekil` hiyerarşisinin elle kurulmuş kaba bir benzeridir. Karşılaştırma için verilmiştir; laboratuvarda yazılmayacaktır.

sekil.c

```c
typedef struct Sekil {
    const char *ad;
    double (*alan)(struct Sekil *);   /* fonksiyon isaretcisi = sanal metot */
} Sekil;

typedef struct {
    Sekil taban;        /* ilk alan taban yapi = elle kurulmus miras */
    double yaricap;
} Daire;

static double daire_alan(Sekil *s) {
    Daire *d = (Daire *) s;          /* elle asagi donusum: TEHLIKELI */
    return 3.14159265 * d->yaricap * d->yaricap;
}
```

Kod 3.20 — C'de elle kurulan sanal metot tablosu

Bu kodda üç kırılganlık vardır: (a) `taban` alanının **ilk** alan olması zorunludur, aksi hâlde dönüşüm bozulur; (b) `daire_alan` içindeki dönüşüm denetlenmez, yanlış tür gelirse bellek bozulur; (c) her yeni tür için işaretçiyi elle bağlamayı unutmak sessiz hataya yol açar. Java'nın dört ilkesi, bu üç riskin tamamını dilin kendisine devreder.

> **ⓘ BİLGİ**
>
> C++ bilenler için not: Java'daki tüm metotlar varsayılan olarak **sanaldır** (`virtual`). C++'ta `virtual` yazmayı unutursanız dinamik bağlama olmaz; Java'da böyle bir risk yoktur. Bunun tersi de doğrudur: Java'da bir metodun ezilmesini engellemek istiyorsanız `final` yazmalısınız.

6. Sık Yapılan Hatalar

6.1. Kapsüllemeyi Yalnızca Alan Gizleme Sanmak

✕ SIK YAPILAN HATA

Alanları `private` yapıp her biri için düşünmeden `getter` ve `setter` yazmak kapsülleme değildir; yalnızca **daha uzun yazılmış açık alandır**. `setBakiye()` metodu varsa, `public double bakiye` ile aynı şeyi biraz daha yavaş yapıyorsunuz demektir.

karsilastirma.java

```java
// YANLIS - kural yok, koruma yok
public class Hesap {
    private double bakiye;
    public double getBakiye() { return bakiye; }
    public void setBakiye(double b) { this.bakiye = b; }   // her deger gecer
}

// DOGRU - kural sinifin icinde, setter yok
public class Hesap {
    private double bakiye;
    public double getBakiye() { return bakiye; }
    public boolean cek(double tutar) {
        if (tutar <= 0 || tutar > bakiye) { return false; }
        bakiye -= tutar;
        return true;
    }
}
```

Kod 3.21 — Yanlış / doğru: kapsülleme

6.2. @Override İşaretleyicisini Yazmamak

✕ SIK YAPILAN HATA

`@Override` yazmadığınızda, metot adını yanlış yazarsanız veya parametre listesini tutturamazsanız derleyici **hiçbir şey söylemez**. Ortaya, üst sınıftaki metodu ezmeyen yepyeni bir metot çıkar ve çok biçimlilik sessizce çalışmaz.

karsilastirma.java

```java
// YANLIS - 'alan' yerine 'Alan'; ezme GERCEKLESMEZ
public class Kare extends Sekil {
    public double Alan() { return kenar * kenar; }    // yeni metot!
}

// DOGRU - @Override derleyiciye denetletir
public class Kare extends Sekil {
    @Override
    public double alan() { return kenar * kenar; }
```

```
10  }
```

Kod 3.22 — Yanlış / doğru: metot ezme

Yukarıdaki yanlış sürümde ayrıca ikinci bir hata daha vardır: `Sekil` soyut `alan()` metodunu bildirdiği için, `Kare` onu doldurmadığından sınıf hiç derlenmez. Bu, soyut sınıfın sağladığı korumadır.

6.3. Soyut Sınıftan Nesne Üretmeye Çalışmak

✕ SIK YAPILAN HATA

`new Sekil("Daire", "kirmizi")` yazamazsınız. Soyut sınıf, gövdesi olmayan metotlar içerdiği için **eksiktir**; ondan nesne üretmek, çalıştırılamayacak bir metoda sahip nesne üretmek demektir.

Derleyici iletisi	Anlamı	Çözüm
`Sekil is abstract; cannot be instantiated`	Soyut sınıftan `new` ile nesne üretilmeye çalışıldı	Somut bir alt sınıf kullanın: `new Daire(...)`
`Kare is not abstract and does not override abstract method alan()`	Alt sınıf, soyut metodu doldurmadı	Eksik metodu yazın veya sınıfı da `abstract` bildirin
`method does not override or implement a method from a supertype`	`@Override` var ama üst türde böyle bir metot yok	Metot adını ve parametre listesini üst sınıfla eşitleyin
`cannot find symbol: method getYaricap()`	Üst tür referansıyla alt türe özgü üye çağrıldı	Önce `instanceof` ile sınayıp dönüşüm yapın
`incompatible types: Sekil cannot be converted to Daire`	Aşağı dönüşüm açıkça yazılmadı	`(Daire) s` biçiminde dönüşüm yazın (sınamadan sonra)

Tablo 3.14 — Bu haftanın konularıyla ilgili sık görülen derleyici iletileri

6.4. Kalıtımı Kod Tekrarını Azaltmak İçin Zorlamak

✕ SIK YAPILAN HATA

İki sınıfta benzer kod olması, aralarında kalıtım kurmak için yeterli gerekçe **değildir**. `Ogrenci extends Dizi` gibi bir tasarım, öğrencinin bir dizi olduğunu iddia eder ve saçmadır. Ölçüt daima şudur: *"Alt sınıf, üst sınıfın kullanıldığı her yerde onun yerine geçebilir mi?"*

6.5. Üst Tür Referansı ile Alt Tür Üyesine Erişmeye Çalışmak

karsilastirma.java

```
1  Sekil s = new Daire(3.0, "kirmizi");
2
3  double a = s.alan();          // DERLENIR   - alan() Sekil'de bildirilmis
4  double r = s.getYaricap();    // DERLENMEZ  - getYaricap() yalnizca Daire'de var
5
6  if (s instanceof Daire) {     // DOGRU YOL
```

```
7       Daire d = (Daire) s;
8       double r2 = d.getYaricap();
9   }
```

Kod 3.23 — Yanlış / doğru: üst tür referansıyla erişim

6.6. Alt Sınıfta super(...) Çağrısını Unutmak

✖ SIK YAPILAN HATA

Üst sınıfta parametresiz yapıcı yoksa ve alt sınıf yapıcısında `super(...)` çağrısı yazılmamışsa, derleyici otomatik `super()` eklemeye çalışır ve **bulamaz**. Sonuç: `constructor Sekil in class Sekil cannot be applied to given types`. Çözüm, uygun parametrelerle `super(...)` çağrısını yapıcının **ilk satırı** olarak yazmaktır.

7. İyi Tasarım İlkeleri

Bu bölüm, dört ilkeyi kullanırken uyulması önerilen pratik kuralları toplar. Laboratuvar değerlendirmesinde bu kurallara uyum da dikkate alınacaktır.

- **Alanlar varsayılan olarak `private` olsun.** Bir alanı dışarıya açmak için gerekçeniz olmalı; kapatmak için gerekçe gerekmez.
- **Her `setter`'ı sorgulayın.** Alanı keyfî bir değere ayarlamak anlamlı bir iş işlemi değilse `setter` yazmayın; işlemi adlandıran bir metot yazın.
- **Ezilen her metoda `@Override` yazın.** Bu, derleyiciye ücretsiz bir denetim yaptırır ve niyetinizi okuyucuya bildirir.
- **Değişkenleri mümkün olan en genel türle bildirin.** `Sekil s = new Daire(...)` , `Daire s = new Daire(...)` bildiriminden daha esnektir.
- **Üst sınıflara ortak olmayan üye koymayın.** `Sekil` sınıfına `yaricap` alanı eklemek, kareyi ve üçgeni anlamsız veri taşımaya zorlar.
- **Soyut sınıfı, ortak veri + ortak davranış varsa seçin.** Yalnızca ortak bir yetenek paylaşılıyorsa arayüz daha uygundur.
- **`instanceof` zincirlerinden kaçının.** Üçüncü `instanceof` sınamasını yazarken durup davranışı sınıflara taşımayı düşünün.
- **Sınıf adları isim, metot adları fiil olsun.** `Sekil`, `BankaHesabi`, `Odeme` — `alan()`, `cek()`, `ode()`.
- **Bir sınıf tek bir sorumluluk taşısın.** `BankaHesabi` para saklar ve kuralları uygular; ekrana rapor basmak onun işi değildir.

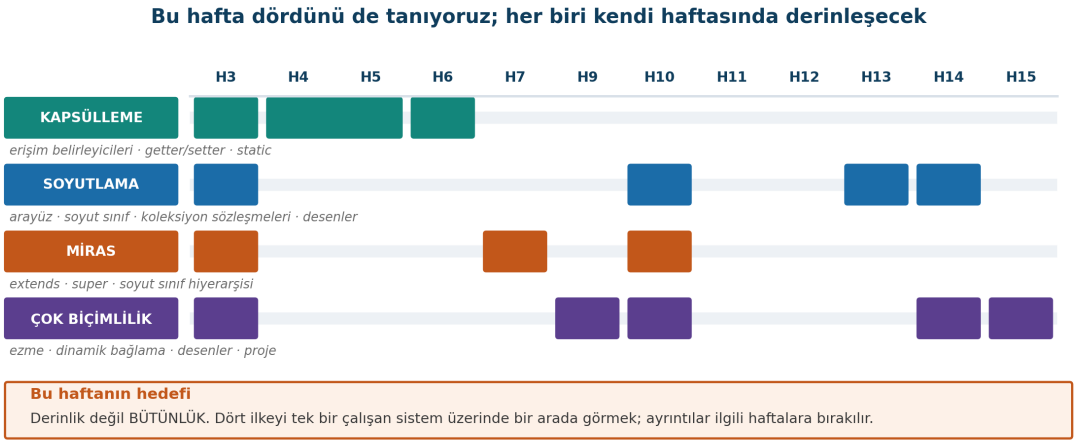
Soru	Yanıt “evet” ise	Yanıt “hayır” ise
“X bir Y'dir” doğal mı?	Kalıtım düşünün (`extends`)	Bileşim veya arayüz düşünün
Ortak **veri** paylaşılıyor mu?	Soyut sınıf uygun	Arayüz yeterli olabilir
Alan dışarıdan keyfî değer alabilir mi?	`setter` yazılabilir	İş metodu yazın, `setter` yazmayın
Aynı `switch` birden çok metotta var mı?	Tür hiyerarşisine geçin	Mevcut yapı kabul edilebilir
Alt sınıf üst sınıfın yerine geçebilir mi?	Kalıtım güvenlidir	Kalıtım kurmayın (tasarım kırılır)

Tablo 3.15 — Tasarım kararları için hızlı kontrol soruları

> **✓ İPUCU**
>
> Bu haftanın en pratik alışkanlığı şudur: bir sınıf yazmaya başlamadan önce **iki cümle** kurun. Birincisi *“Bu sınıf neyi bilir?”* (alanlar), ikincisi *“Bu sınıf ne yapabilir?”* (metotlar). Cümleler kurulamıyorsa sınıf henüz düşünülmemiştir.

8. Bir Sonraki Haftaya Köprü

Bu hafta dört ilkeyi tanıdık. Bundan sonraki on hafta, bu dört ilkenin ayrıntılarını açacak. Aşağıdaki şekil dönem haritasını gösterir.



Şekil 3.9 — Dört ilkenin dönem boyunca derinleşme takvimi

Hafta	Konu	Bu haftanın hangi kısmını derinleştirir?
Hafta 4	Kapsülleme ve erişim belirleyicileri	Bölüm 4.1 — dört erişim düzeyi, doğrulamalı setter, `final` alan, değişmez sınıf
Hafta 5	Yapıcı ve yıkıcı yöntemler	Bölüm 3.3 — `super()` zinciri, kopya yapıcı, nesne başlatma sırası
Hafta 6	Statik ve nesne üyesi değişkenler	Bölüm 4.1 — `static final` sabitler (`GUNLUK_CEKIM_LIMITI` gibi)
Hafta 7	Miras	Bölüm 4.3 — `super`, `protected`, elmas problemi, IS-A / HAS-A kararı
Hafta 9	Çok biçimlilik	Bölüm 4.4 ve 4.5 — dinamik bağlama, `equals`/`hashCode`, güvenli dönüşüm
Hafta 10	Soyut sınıflar ve arayüzler	Bölüm 4.2 ve 4.6 — `default` metotlar, çoklu arayüz, `Comparable`
Hafta 14	Tasarım desenleri	Dört ilkenin birlikte kullanıldığı hazır çözüm kalıpları

Tablo 3.16 — Bu haftanın konularının dönem içindeki devamı

Gelecek hafta **kapsüllemeyi** derinleştireceğiz. Bu hafta `private` ve `public` düzeylerini kullandık; Hafta 4'te dört erişim düzeyinin tamamını (`private`, varsayılan/paket, `protected`, `public`) ve hangisinin ne zaman seçileceğini göreceğiz. `BankaHesabi` sınıfını daha da sıkılaştıracak, değiştirilemez (immutable) sınıf tasarımını öğreneceğiz.

9. Haftanın Özeti

- Tek sınıflı programlarda yeterli olan araçlar, **birbirine benzeyen ama aynı olmayan** varlıklar aynı programda buluştuğunda yetmez.

- **Tip kodu + `switch`** yaklaşımı üç kusur üretir: sessiz yanlış sonuç, ölü alanlar ve yeni tür eklerken çok noktalı değişiklik.
- **Kapsülleme**, veriyi ve onu değiştiren kuralı aynı sınıfta birleştirir. En güçlü biçimi, `setter` yerine anlamlı iş metotları yazmaktır.
- **Soyutlama**, "ne" ile "nasıl"ı ayırır. Java'da soyut sınıf ve arayüz bu ayrımın iki aracıdır.
- **Miras**, ortak veri ve davranışı üst sınıfta bir kez yazmayı sağlar; `extends` ile kurulur ve `IS-A` ilişkisini ifade eder.
- **Çok biçimlilik**, aynı çağrının nesnenin gerçek türüne göre farklı gövde çalıştırmasıdır; `switch` zincirlerini ortadan kaldırır.
- Bir referansın **bildirim türü** hangi üyelerin çağrılabileceğini, **gerçek türü** hangi gövdenin çalışacağını belirler.
- Kalıtım *"bu nedir?"*, arayüz *"bu ne yapabilir?"* sorusunu yanıtlar. Java'da tek üst sınıf, çok arayüz mümkündür.

→ BİR SONRAKİ HAFTA

Hafta 4'te kapsüllemeyi tek başına ve derinlemesine ele alacağız: dört erişim düzeyi, doğrulama mantığının setter'a taşınması, `final` alanlar, değişmez (immutable) sınıf tasarımı ve paket kavramı. Bu haftanın `BankaHesabi` ve `Sekil` sınıflarını yanınızda getirin; onları sıkılaştırarak başlayacağız.

10. Alıştırmalar

Alıştırmalar üç gruba ayrılmıştır: kavram soruları, kod okuma / hata bulma ve programlama. Zorluk göstergesi: ●○○ kolay, ●●○ orta, ●●● zor.

10.1. Kavram Soruları

Alıştırma 1 ●○○

Dört temel ilkeyi adlandırın ve her biri için **birer cümlelik** tanım yazın. Ardından her ilkenin bu haftanın hangi ders örneğinde görüldüğünü belirtin.

Alıştırma 2 ●○○

"Kapsülleme, alanları `private` yapmaktır" cümlesi neden eksik bir tanımdır? `setBakiye()` metodu olan bir sınıfın kapsüllenmiş sayılıp sayılamayacağını gerekçelendirin.

Alıştırma 3 ●●○

Bir referansın bildirim türü ile gerçek türü arasındaki farkı açıklayın. `Sekil s = new Kare(4.0, "mavi");` bildiriminden sonra `s.alan()` ve `s.getKenar()` çağrılarından hangisi derlenir, hangisi derlenmez? Neden?

Alıştırma 4 ●●○

`extends` ile `implements` arasındaki farkı, "IS-A" ve "CAN-DO" kavramlarını kullanarak açıklayın. Bir sınıfın neden yalnızca tek bir sınıftan miras alabildiğini, elmas problemine değinerek yazın.

10.2. Kod Okuma ve Hata Bulma

Deneme.java

```java
abstract class Arac {
    abstract void calistir();
}

class Otomobil extends Arac {
    void surus() { }
}

public class Deneme {
    public static void main(String[] args) {
        Arac a = new Arac();
    }
}
```

Kod 3.24 — Alıştırma 5 için kod listesi

Alıştırma 5 ●●○

Yukarıdaki kod parçası derlenmez ve **iki ayrı** hata içerir. Her iki hatayı da bulun, derleyicinin vereceği iletileri tahmin edin ve kodu düzeltin.

İpucu: Birinci hata `Otomobil` sınıfında, ikincisi `main` metodundadır. Bölüm 6.3'teki derleyici iletileri tablosu yardımcı olacaktır.

Alıştırma 6 ●●○

`SekilKotu.java` programında `"ucgen"` türü için `alan()` metodu `0.0` döndürüyor. Bu davranışı bir istisna fırlatarak değiştirseydik hangi sorunu çözmüş, hangi sorunu çözmemiş olurduk? Tür hiyerarşisi çözümünün farkını açıklayın.

Rapor.java

```
1  void yazdir(Object o) {
2      if (o instanceof Daire) {
3          System.out.println("Daire, alan = " + ((Daire) o).alan());
4      } else if (o instanceof Kare) {
5          System.out.println("Kare, alan = " + ((Kare) o).alan());
6      } else if (o instanceof Ucgen) {
7          System.out.println("Ucgen, alan = " + ((Ucgen) o).alan());
8      } else {
9          System.out.println("Bilinmeyen tur");
10     }
11 }
```

Kod 3.25 — Alıştırma 7 için kod listesi

Alıştırma 7 ●●●

Yukarıdaki metot çok biçimlilik kullanmıyor. Hangi tasarım kusurunu barındırdığını yazın; bu kusurun Bölüm 2'deki `switch` sorunuyla ilişkisini açıklayın ve metodu `instanceof` kullanmadan **iki satırda** yeniden yazın.

İpucu: Çözüm, davranışı çağıran taraftan sınıfların içine taşımaktır. Parametre türünü değiştirmeyi düşünün.

10.3. Programlama

Alıştırma 8 ●○○

`Sekil` hiyerarşisine `Dikdortgen` adında yeni bir alt sınıf ekleyin. Sınıf `genislik` ve `yukseklik` alanlarını tutsun, `alan()` ve `cevre()` metotlarını doldursun. `SekilDemo` programına **dokunmadan** yeni şeklin raporda göründüğünü kanıtlayın.

İpucu: Yalnızca yeni dosyayı yazıp diziye bir öğe eklemeniz yeterli olmalıdır.

Alıştırma 9 ●●○

`BankaHesabi` sınıfına `havaleGonder(BankaHesabi hedef, double tutar)` metodu ekleyin. Metot, `cek()` başarılı olursa hedef hesaba `yatir()` yapsın; başarısızsa hiçbir değişiklik olmamalıdır. Sınıfın dışına hiçbir kural sızdırmayın.

İpucu: `cek()` metodunun `boolean` döndürdüğünü hatırlayın.

Alıştırma 10 ●●○

`Odeme` arayüzünü gerçekleyen dördüncü bir sınıf yazın: `HediyeCeki`. Çekin bir tutarı olsun ve **kısmi kullanıma izin vermesin** — yalnızca tutarın tamamına eşit bir ödeme kabul edilsin. `OdemeDemo` içindeki `Kasa` sınıfına dokunmadan yeni yöntemin çalıştığını gösterin.

Alıştırma 11 ●●○

Bir `Calisan` üst sınıfı ve ondan türeyen `Muhendis`, `Yonetici` sınıfları yazın. `maasHesapla()` metodu her alt sınıfta farklı çalışsın (örneğin yöneticiye yönetim tazminatı eklensin). `Calisan[]` dizisi üzerinde tek döngüyle toplam maaş hesaplayın.

İpucu: `Sekil` / `SekilDemo` ikilisini örnek alın.

Alıştırma 12 ●●●

Bir `Kutuphane` sınıfı tasarlayın; içinde `Yayin` soyut sınıfı, `Kitap` ve `Dergi` alt sınıfları ve `Oduncalinabilir` arayüzü bulunsun. Dergiler ödünç verilemesin. Ödünç verme kuralı yalnızca `Kutuphane` sınıfının içinde olsun ve dört ilkenin hangi satırda kullanıldığını yorum satırlarıyla işaretleyin.

İpucu: Bu, laboratuvar Görev 5'in genişletilmiş sürümüdür.

11. Öz Değerlendirme

Laboratuvara girmeden önce aşağıdaki maddelerin tamamını işaretleyebiliyor olmalısınız. İşaretleyemediğiniz her madde için ilgili bölüme dönün.

Öz Değerlendirme Kontrol Listesi

- ☐ Dört temel ilkeyi adlandırabiliyor ve her birinin çözdüğü problemi söyleyebiliyorum. (Bölüm 3.1)
- ☐ Tip kodu + `switch` yaklaşımının üç kusurunu örnekle açıklayabiliyorum. (Bölüm 2.3)
- ☐ Bir sınıfın alanlarını `private` yapıp iş kurallarını metotlara taşıyabiliyorum. (Bölüm 4.1)
- ☐ `abstract` sınıf ile `interface` arasındaki farkı ve ne zaman hangisinin seçileceğini biliyorum. (Bölüm 4.2 ve 4.6)
- ☐ `extends` ile miras kurabiliyor, `super(...)` çağrısını doğru yere yazabiliyorum. (Bölüm 3.3)

- ☐ `@Override` işaretleyicisinin ne işe yaradığını ve neden yazılması gerektiğini biliyorum. (Bölüm 6.2)
- ☐ Üst tür referansı ile alt tür nesnesi kullanarak çok biçimli bir dizi yazabiliyorum. (Bölüm 4.4)
- ☐ Bildirim türü ile gerçek tür arasındaki farkı örnek üzerinde gösterebiliyorum. (Bölüm 4.5)
- ☐ `instanceof` ile güvenli aşağı dönüşüm yazabiliyorum ve neden gerekli olduğunu biliyorum. (Bölüm 4.5)
- ☐ Verilen bir kod parçasında hangi ilkenin kullanıldığını tespit edebiliyorum. (Bölüm 4.7)

12. Kaynaklar

12.1. Ders Kitapları

Kaynak	İlgili bölüm	Ne için?
Deitel & Deitel, *Java How to Program*	Bölüm 8 (sınıflar), Bölüm 9 (miras), Bölüm 10 (çok biçimlilik)	Bol örnekli, adım adım anlatım; bu haftanın tamamını kapsar
Cay Horstmann, *Core Java, Volume I*	Bölüm 4 (nesneler ve sınıflar), Bölüm 5 (miras)	Kavramsal derinlik ve dilin gerekçeleri
Herbert Schildt, *Java: The Complete Reference*	Bölüm 8 (miras), Bölüm 9 (paketler ve arayüzler)	Başvuru niteliğinde; sözdizimi ayrıntıları
Joshua Bloch, *Effective Java* (3. baskı)	Madde 18 (kalıtım yerine bileşim), Madde 20 (soyut sınıf yerine arayüz)	İleri düzey; tasarım kararlarının gerekçeleri

Tablo 3.17 — Bu haftanın konuları için önerilen kitap bölümleri

12.2. Çevrimiçi Kaynaklar

- Oracle Java Tutorials — *Object-Oriented Programming Concepts*: `docs.oracle.com/javase/tutorial/java/concepts/`
- Oracle Java Tutorials — *Inheritance*: `docs.oracle.com/javase/tutorial/java/IandI/subclasses.html`
- Oracle Java Tutorials — *Abstract Methods and Classes*: `docs.oracle.com/javase/tutorial/java/IandI/abstract.html`
- Java Language Specification, §8.1.4 *Superclasses and Subclasses* — dilin resmî tanımı; kural tartışmalarında son sözü söyler.
- Oracle JDK 21 API belgeleri — `java.lang.Object` sınıfı (`toString`, `equals`, `getClass` açıklamaları).

12.3. Bu Haftanın Kod Dosyaları

Dosya	Konu	Bölüm
`SekilKotu.java`	Tip kodu + `switch` yaklaşımı ve kusurları	2.1, 2.2
`Sekil.java`	Soyut üst sınıf; kapsülleme, soyutlama, miras	3.2
`Daire.java`	Birinci alt sınıf; `extends`, `super`, `@Override`	3.3
`Kare.java`	İkinci alt sınıf; aynı sözleşme, farklı formül	3.3

Dosya	Konu	Bölüm
`Ucgen.java`	Sonradan eklenen tür; genişletilebilirlik kanıtı	3.4
`SekilDemo.java`	Dört ilkenin tek programda gösterimi	4.4, 4.7
`BankaHesabi.java`	Kapsülleme; kural verinin yanında	4.1
`KapsullemeDemo.java`	Kapsüllemenin engellediklerinin kanıtı	4.1
`Odeme.java`	Arayüz; yalnızca sözleşme	4.2
`OdemeDemo.java`	Üç gerçekleme + gerçeklemeyi bilmeyen `Kasa`	4.2

Tablo 3.18 — Ders örneklerinin dosya listesi (`Hafta_03/Java_Kodlari/ders/`)

12.4. İletişim

Rol	Ad	Görev
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE	Teori dersi, sınav, dönem projesi
Laboratuvar	Arş. Gör. Betül MUMCU	Laboratuvar oturumu, föy değerlendirmesi